What is JavaScript?

* JavaScript is a powerful and versatile programming language

* JavaScript is an interpreted, client-side, event-based, object oriented scripting language

* It helps to create dynamic interactivity web pages.

* It was originally developed by Netscape

V8 Engine

* The v8 engine refers to the JavaScript engine

* Developed by Google for the Chrome browser.

* It is the core component responsible for executing JavaScript code

* The engine is implemented in C++ for speed and performance.

* The V8 engine uses Just-In-Time (JIT) compilation

Compiled Execution (V8's Approach)

* It compiles the JavaScript into machine code first (Instead of directly executing the code line by line)

* The machine code is stored and reused during the program's execution.

* This avoids the overhead of interpreting the same code multiple times.

* Optimized performance: The engine identifies frequently used parts of the code (called hot code) and optimizes it further during execution.

What is Node.js?

* Node runs on the server, not in the browser.

* It is built on Chrome's V8 JavaScript engine

* Node.js is an open-source, cross-platform JavaScript runtime environment.

* The NodeJS is a single-threaded environment, which means that it runs one action at a time.

* Node.js uses an event-driven, non-blocking I/O model, making it efficient and suitable for real-time applications.

Single-threaded environment

* JavaScript code runs on a single main thread.

* Asynchronous tasks (e.g., file reads, API calls) are offloaded to the event loop.

* Non-blocking I/O ensures high performance for handling multiple requests.

* CPU-intensive tasks can use Worker Threads for multi-threading if needed.

Event Loop

* The event loop is a core concept in Node.js

* It is responsible for handling asynchronous operations

* It’s enabling non-blocking I/O

* It allows Node.js to perform tasks, execute callbacks to the system while continuing to execute other code.

* Node.js uses a single-threaded event loop model

* Enables high performance in Node.js by handling thousands of concurrent connections on a single thread.

* Powers asynchronous programming in Node.js

Setting up Node js Environment

* Install Node

* Package Manager (npm or yarn)

* Code Editor (Visual Studio Code)

* Node Version Manager (nvm) Optional

How to install the node

Node.js provides the runtime environment for running JavaScript code.

Steps:

* Visit the Node.js website (https://nodejs.org)

* Download the recommended LTS(Long-Term Support) version

* Click on "Next," "Next," "Next," and install Node.js

Checking Node.js Installation:

* Open a terminal or command prompt cmd

* Type the following command : node -v

* Press Enter

* Example Output: "v20.11.1" (version number may vary)

Read-Eval-Print Loop (REPL)

* It is an interactive programming environment that

* takes single user inputs (Read),

* evaluates them (Eval),

* prints the result (Print),

* and then loops back to wait for more user input (Loop).

Using the Node.js REPL

Starting Node.js REPL:

* Open a terminal or command prompt cmd

* Type the following command: node

* Press Enter

* The REPL prompt (>) indicating that the REPL environment is ready to accept your commands.

Simple Programs using REPL

Simple Calculations:

> 3 + 3

Create Javascript Variables:

```
> let marks = 80
```

Create Functions:

```
> function add( a + b ) {  return a + b; }
```

Multiline Functions:

```
> function add( a + b ) {
...   return a * b;
... }
```

Accessing Node.js Modules:

```
> const fs = require('fs')
> fs.readFileSync('./example.txt', 'utf8')
'Hello, World!'
```

REPL commands

.help Displays help for the REPL commands.

.exit Exit the REPL

.help Print this help message

.editor Allowing you to write multi-line JavaScript code

.break Sometimes you get stuck, this gets you out

```
> {
... for (let i = 0; i < 10; i++) {
... .break
```

.clear Alias for .break

```
> {
... for (let i = 0; i < 10; i++) {
```

... .clear

.load Load JS from a file into the REPL session

.load main.js

.save Save all evaluated commands in this REPL session to a file

.save session.js

Create First Project in VS Code

1) Create a empty folder

2) Load folder into VScode

3) Open terminal

4) enter this command

> npm init -y

5) Run the application

> node main.js

Global Objects

*console: **

Used to print to stdout and stderr.

console.log('Hello, World!');

console.error('Error message');

_ _dirname:

Returns the directory name of the current module.

console.log(__dirname);

_ _filename:

Returns the file name of the current module.

console.log(__filename);

process:

Provides information about the current Node.js process.

console.log(process.env); // Environment variables

console.log(process.argv); // Command line arguments

process.env:

Contains the user environment.

console.log(process.env.NODE_ENV);

// Accessing the NODE_ENV environment variable

Timers Objects

Timers are functions that allow you to execute code at a specified time interval or after a certain delay. Timers are not perfectly accurate. They can be delayed due to the single-thread.

The two primary functions for creating timers in JavaScript are

1) setTimeout 2) setInterval

corresponding functions to stop these timers:

1) clearTimeout 2) clearInterval

setTimeout

used to execute a function once after a specified delay.

```
let timeoutID = setTimeout(function() {
  console.log("delayed by 2 seconds");
}, 2000);
```

clearTimeout

To cancel a timeout set by setTimeout, you use clearTimeout

```
clearTimeout(timeoutID);
```

setInterval

used to repeatedly execute a function at a specified interval.

```
let intervalID = setInterval(function(){
 console.log("repeats every 3 seconds");
}, 2000);
```

clearInterval

To stop an interval set by setInterval, you use clearInterval.

```
clearInterval(intervalID);
```

Modules

1. JavaScript modules provide a way to break up your code into separate files

2. Modules are making it more organized, maintainable, and reusable.

3. Modules are imported from external files with the import statement.

4. Modules also rely on type="module" in the <script> tag.

5. A web-page opened via the file:// protocol cannot use import / export.

* JavaScript modules are reusable pieces of code

* It can be imported and exported within different files or programs.

There are two primary types of modules in JavaScript:

1. ES6 Modules (ECMAScript 2015)

2. CommonJS Modules (used primarily in Node.js)

1) ES6 Modules

ES6 introduced a standardized module system in JavaScript, which is now widely used in modern web development.

2) CommonJS Modules

CommonJS is primarily used in Node.js for server-side development.

1. Exporting from a Module

(use module.exports to export a module)

```
function add(a, b) {
  return a + b;
}
function subtract(a, b) {
  return a - b;
}
module.exports = {
  add,
  subtract
};
```

2. Importing from a Module (use require to import a module)

```
const math = require ('./math.js');
console.log(math.add(2, 3)); // 5
console.log(math.subtract(5, 3)); // 2
```

Custom Modules

Create a new file for your module, e.g., common.js.

```
function add (a, b) {
  return a + b;
```

```
}

const userWelcomeMsg = (name) => {

  return Hello, ${name}!;

}

module.exports = {

  add,

  userWelcomeMsg

};
```

index.js

```
const user = require('./common');

const greet = user.userWelcomeMsg('Ram');

console.log(greet); // Hello, Ram!
```

(or)

```
const {add, userWelcomeMsg} = require('./common');

const sum = add(10,20);

console.log(sum); // 30
```

Built-In Modules

os Provides operating system-related utility methods and properties.

path Provides utilities for working with file and directory paths.

fs (File System) Provides an API for interacting with the file system.

Http Allows Node.js to transfer data over HTTP.

events Provides a way of working with events.

url Provides utilities for URL resolution and parsing.

querystring Provides utilities for parsing and formatting URL query strings.

crypto Provides cryptographic functionality that includes a set of wrappers for OpenSSL's hash, HMAC, cipher, decipher, sign, and verify functions.

os Module

os Provides operating system-related utility methods and properties.

```
const os = require('os');

console.log('Version:', os.version());

console.log('Type:', os.type());
```

(or)

```
console.log('OS Platform:', os.platform());

console.log('OS CPU architecture:', os.arch());

console.log('Total memory:', os.totalmem());

console.log('Free memory:', os.freemem());

console.log('Cpus:', os.cpus());
```

path Module

```
const path = require('path');
```

Directory name of the current file

```
console.log(path.dirname(__filename))

console.log(path.basename(__filename))

console.log(path.extname(__filename));
```

Object with parsed path details

```
console.log(path.parse(__filename));
```

Combine paths

```
const directory = '/main/images';

const fileName = 'avatar/user.jpg';

console.log(path.join(directory, fileName));// '/main/images/avatar/user.jpg'
```

Check if path is absolute

```
console.log(path.isAbsolute('/foo/bar')); // true

console.log(path.isAbsolute('foo/bar'));  // false
```

Fs (File System) Module

1. It provides a way to interact with the file system.

2. It allows to perform various file operations like reading, writing, deleting, and more.

3. This module can be used synchronously or asynchronously.

Commonly used methods of the fs module:

* Reading Files (fs.readFile)

* Writing Files (fs.writeFile)

* Appending to Files (fs.appendFile)

* Deleting Files (fs.unlink)

* Renaming Files (fs.rename)

* Checking File Existence (fs.access)

* Creating Directories (fs.mkdir)

* Reading Directories (fs.readdir)

Reading Files

```
fs.readFile('sample.txt', 'utf8', (err, data) => {

  if (err) {

     console.error(err);

     return;

  }

  console.log(data);
```

```
});
```

Writting File

```
fs.writeFile('sample.txt', 'Hello, world!', (err) => {
  if (err) {
    console.error(err);
    return;
  }
  console.log('File written successfully');
});
```

Appending to Files

```
fs.appendFile('sample.txt', '\nAppended new text.', (err) => {
  if (err) {
    console.error(err);
    return;
  }
  console.log('File appended successfully');
});
```

Deleting Files

```
fs.unlink('sample.txt', (err) => {
  if (err) {
    console.error(err);
    return;
  }
  console.log('File deleted successfully');
```

```
});
```

Renaming Files

```
fs.rename('oldFileName.txt', 'newFileName.txt', (err) => {
  if (err) {
    console.error(err);
    return;
  }
  console.log('File renamed successfully');
});
```

Checking File Existence

```
fs.access('sample.txt', fs.constants.F_OK, (err) => {
  console.log(${err ? 'File does not exist' : 'File exists'});
});
```

Creating Directories

```
fs.mkdir('SampleDir', { recursive: true }, (err) => {
  if (err) {
    console.error(err);
    return;
  }
  console.log('Directory created successfully');
});
```

Reading Directories

```
fs.readdir('SampleDir', (err, files) => {
  if (err) {
```

```
    console.error(err);
    return;
  }
  console.log('Directory contents:', files);
});
```

http Module

It is used for creating HTTP servers and making HTTP requests in Node.js.

Creating an HTTP Server

```
const http = require('http');
const server = http.createServer((req, res) => {
  res.write('This is first server application');
  res.end();
});
server.listen(5500);
```

Check server

Open browser and enter the port

http://localhost:5500/

Creating an HTTP Server

```
const http = require('http');
const server = http.createServer((req, res) => {
        res.writeHead(200, { 'Content-Type': 'text/plain' });//application/json
        res.end('Hello, World!\n');
});
server.listen(5500, '127.0.0.1', () => {
```

```
        console.log(Server running at http://${hostname}:${port}/);
});
```

Check server

Open browser and enter the port

http://localhost:5500/

Handling Routes with Plain Text

```
const http = require('http');
const server = http.createServer((req, res) => {
        res.writeHead(200, { 'Content-Type': 'text/plain' });
        if (req.url === '/') {
                res.end('Home Page\n');
        } else if (req.url === '/about') {
                res.end('About Page\n');
        } else {
                res.end('404 Not Found\n');
        }
});
server.listen(3000, '127.0.0.1', () => {
  console.log(Server running at http://127.0.0.1:3000/);
});
```

Handling Routes with JSON Responses

```
const http = require('http');
```

```
const server = http.createServer((req, res) => {
	res.writeHead(200, { 'Content-Type': 'application/json' });
	if (req.url === '/data') {
		const data = {
		message: 'Hello, World!',
		timestamp: new Date(),
		};
		res.end(JSON.stringify(data));
	} else {
		res.writeHead(404, { 'Content-Type': 'application/json' });
		res.end(JSON.stringify({ error: '404 Not Found' }));
	}
});

server.listen(3000, '127.0.0.1', () => {
  console.log(Server running at http://127.0.0.1:3000/);
});
```

events Module

Event Emitter

1. EventEmitter is a class.

2. It is a part of the events module

3. It is used to handle and manage events.

4. It allows to create, emit, and listen to custom events

The events module allows to work with event-driven programming.

It provides a way to handle asynchronous events, such as I/O operations, timers, and custom events.

Steps to create Events

1. Import events Module

2. Create an instance using the EventEmitter class

3. Use on method to attach a listener to an event

4. Use emit method to trigger an event

```
const EventEmitter = require('events');

const myEmitter = new EventEmitter();

myEmitter.on('message', () => {

  console.log('Welcome to Hyderabad');

});

myEmitter.emit('message');
```

Event Methods

1. on (eventName, listener)

2. once (eventName, listener)

3. emit (eventName, [...args])

4. removeListener (eventName, listener)

5. removeAllListeners ([eventName])

6. listeners (eventName)

7. listenerCount (eventName)

8. eventNames()

on(eventName, listener)

Adds a listener to the end of the listeners array for the specified event.

```
emitter.on('event', (message) => {
	console.log(message);
});
```

(or)

```
const listener1 = (message) => console.log('Listener 1:', message);
emitter.on('event', listener1);


emitter.emit('event', 'Hello World!');
emitter.emit('event', 'Hello World!');
```

once(eventName, listener)

* Adds a one-time listener for the event.

* It is invoked only the first time the event is fired, after which it is removed.

```
emitter.once('event', (message) => {
	console.log(message);
});
emitter.emit('event', 'Hello World!');
emitter.emit('event', 'Hello Again!'); // No output
```

emit(eventName, [...args])

Synchronously triggers the event handlers that are listening for the event.

```
emitter.emit('event', 'Hello World!');
emitter.emit('event', 'Hello Again!');
```

removeListener(eventName, listener)

Removes a specific listener from the listener array for the event named eventName.

```
const EventEmitter = require('events');
const emitter = new EventEmitter();
const messageListener = () => { console.log('Welcome to Hyderabad') };
myEmitter.on('message', messageListener);
myEmitter.removeListener('message', messageListener);
myEmitter.emit('message');
```

removeAllListeners([eventName])

Removes all listeners, or those of the specified event.

```
emitter.on('event', (message) => {
	console.log('First:',message);
});
emitter.on('event', (message) => {
	console.log('Second:',message);
});
emitter. removeAllListeners ('event');
emitter.emit('event', 'Hello World!');
```

listeners(eventName)

Returns a copy of the array of listeners for the event named eventName.

```
const listener1 = (message) => console.log('Listener 1:', message);
const listener2 = (message) => console.log('Listener 2:', message);
```

```
emitter.on('event', listener1);

emitter.on('event', listener2);


const listeners = emitter.listeners('event');

console.log(listeners); //[[Function: listener1],[Function: listener2]]

console.log(listeners.length); // Output: 2
```

listenerCount(eventName)

Returns the number of listeners listening to the event named eventName.

```
emitter.on('event', (message) => {

  console.log('Listener:', message);

 });


const count = emitter.listenerCount('event');

console.log(count); // Output: 1
```

eventNames()

Returns an array listing the events for which the emitter has registered listeners.

```
const listener1 = (message) => console.log('Listener 1:', message);

const listener2 = (message) => console.log('Listener 2:', message);

emitter.on('event1', listener1);

emitter.on('event2', listener2);

const eventNames = emitter.eventNames();

console.log(eventNames); // Output: [ 'event1', 'event2' ]
```

url Module

* Modern URL handling uses the URL class.

* It provides utilities for URL resolution and parsing.

* It can be used to parse, format, resolve, and validate URLs.

Steps to create url

1) Import url Module

```
const { URL } = require('url');
```

2) Parsing a URL

```
const urlString = 'https://www.xyz.com:8080/searchq=nodejs&oq=nodejs&gs_lcrp=EgZjaHJvbWU#hash';

const parsedUrl = new URL(urlString);

url.username = 'myname' // modify the URL

console.log(parsedUrl);
```

Formatting a URL

It takes an URL object and returns a formatted URL string.

```
const obj = {
  protocol: 'https:',
  hostname: 'www.google.com',
  port: 8080,
  pathname: '/search',
  query: {
    q: 'nodejs',
    oq: 'nodejs'
  },
  hash: '#hash'
};
```

```
const formattedUrl = url.format(obj);

console.log(formattedUrl);
```

o/p: https://www.google.com:8080/search?q=nodejs&oq=nodejs#hash

Resolving URLs

It resolves the target URL relative to the base URL.

```
const baseUrl = 'https://www.google.com:8080';

const relativeUrl = '/search.html?q=nodejs&oq=nodejs#hash';

const resolvedUrl = url.resolve(baseUrl, relativeUrl);

console.log(resolvedUrl);
```

URLSearchParams API

* It provides utility methods to work with the query string of a URL.

* If you pass a full URL, it will not parse the query parameters correctly.

Creating a URLSearchParams Object

```
const { URLSearchParams } = require('url');

const params = new URLSearchParams('q=nodejs&oq=nodejs');

console.log(params.get('q')); // nodejs

console.log(params.get('oq')); // nodejs
```

Adding and Deleting Parameters

```
const { URLSearchParams } = require('url');

const params = new URLSearchParams();

params.append('q', 'node');

params.append('oq', 'nodejs');

console.log(params.toString()); // Output: q=nodejs&oq=nodejs
```

```
params.delete('q');

console.log(params.toString()); // Output: oq=nodejs
```

WHATWG URL API

* It is a modern, standardized API for working with URLs in JavaScript.

* It is a standardized interface for parsing, constructing, and manipulating URLs across web browsers and other environments like Node.js

Basic Parsing

```
const { URL } = require('url');

const myURL = new URL('https://www.google.com:8080/search?q=nodejs');

console.log(myURL.href);           // Output: fullurl
```

Reading Components

```
console.log(url.protocol);  // 'https:'
```

Modifying URL Components

```
myURL.pathname = '/expressjs';

myURL.searchParams.append('oq', 'nodejs');
```

querystring Module

* It provides utilities for URL resolution and parsing.

* It can be used to parse, format, resolve, and validate URLs.

Steps to create url

Import url Module

```
const querystring = require('querystring');
```

Parsing a URL

```
const queryString = 'q=nodejs&oq=nodejs&gs_lcrp=EgZjaHJvbWU';

const parsed = querystring.parse(queryString);
```

console.log(parsed);

// Output: { q: 'nodejs', oq: 'nodejs', gs_lcrp: 'EgZjaHJvbWU' }

const querystring=require(querystring);

Stringifying an Object

const obj = { q: 'nodejs', oq: 'nodejs', gs_lcrp: 'EgZjaHJvbWU' };

const queryString = querystring.stringify(obj);

console.log(queryString); // 'q=nodejs&oq=nodejs&gs_lcrp=EgZjaHJvbWU'

Encoding a Query String

const escaped = querystring.escape('q=nodejs&oq=nodejs');

console.log(escaped); // q%3Dnodejs%26oq%3Dnodejs

Decoding a Query String

const unescaped = querystring.unescape('q%3Dnodejs%26oq%3Dnodejs');

console.log(unescaped); // q=nodejs&oq=nodejs

crypto Module

* It provides cryptographic functionality. The hash is a one-way function, meaning once the data is hashed, it cannot be "reversed" to get the original input.

* It includes a set of wrappers for OpenSSL's hash, HMAC, cipher, decipher, sign, and verify functions.

Steps to use Crypto

Import url Module

const crypto = require('crypto');

Create Hashing

const hash = crypto.createHash('sha256');//Secure Hash Algorithm 256-bit

hash.update('Hello World');

const digest = hash.digest('hex');

```
console.log(digest);
//a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
```

Create and Serve an HTML page using Node.js

* Steps to Create and Serve an "Home" HTML Page in Node.js:

* Create a simple HTML file (like an "home.html" page).

* Set up a Node.js server to serve this HTML file.

```
const http = require('http')
const fs = require('fs')
const server = http.createServer((req, res)=>{
  if(req.url==='/'){
    fs.readFile("home.html",(err, data)=>{
      res.writeHead(200, { 'Content-Type': 'text/html' });
      res.end(data);
    })
  }
})
server.listen(5001, console.log('Server Created'))
```

EXPRESS JS

Express JS

1. Express.js is a Node.js web application framework

2. It provides tools for building web applications and APIs.

3. It provides a robust set of features to develop web and mobile applications.

4. It is used to build single-page, multi-page, and hybrid web applications.

express js Installation:

* Create a folder

* Open a terminal or command prompt cmd

* Type the following command: npm init -y

* Press Enter

* Type the following command: npm install express

(or)

If you install express along with nodemon:

npm install express nodemon

Creating a Basic Server

```
const express = require('express');

const app = express();

app.get('/', (req, res) => {

        res.send('Hello World!');

});

app.listen(3000, () => {

 console.log(Server is running on http://localhost:3000);

});
```

Run the app node server.js

API request methods (or) verbs

1. Each method has a specific purpose

2. It is used to convey the intent of the API request.

GET: Retrieve information or data from the server.

POST: Create a new resource or submit data to the server.

PUT: Update an existing resource with new data.

PATCH: Partially update an existing resource with new data.

DELETE: Remove a resource from the server.

Routing

* Routing is a mechanism defined and handled by the application's endpoints (routes).

* Routes determine how an application responds to a client request

Steps to create routes:

1) Defining Routes: using the app.METHOD() functions, where METHOD is an HTTP method such as get, post, put, delete, etc.

2) Route Handlers: It handles requests to a route. They have access to the request (req), response (res), and next middleware function.

Basic Routing

1. Routes are defined using the methods

2. These methods are provided by the Express application object

Ex:

app.get()

app.post()

app.put()

app.patch()

app.delete()

```
const express = require('express');
```

```
const app = express();

app.get('/', (req, res) => {

  res.send('Hello, World!');

});

app.post('/submit', (req, res) => {

  res.send('Form submitted');

});

app.put('/update/:id', (req, res) => {

  res.send(Update item with ID: ${req.params.id});

});

app.delete('/delete/:id', (req, res) => {

  res.send(Delete item with ID: ${req.params.id});

});

app.listen(3000, () => {

  console.log('Server is running on port 3000');

});
```

app.all()

* app.all will handle all HTTP methods for the specified path.

* It is typically used for tasks like logging, authentication, or setting response headers

* It can also be used for generic route handling when you need a catch-all handler for a route.

```
const express = require('express');

const app = express();

app.all('/user', (req, res) => {

  console.log('Request method:', req.method);
```

```
  res.send('This is a user route with any HTTP method.');
});
app.listen(4000, () => {
  console.log('Server running on port 4000');
});
```

Route Parameters

* They are placeholders in the URL.

* It can be used to capture values.

* Defined with a colon (:) in the route path.

```
app.get('/user/:id', (req, res) => {
                const userId = req.params.id;
                res.send(User ID: ${userId});
});
```

Query Strings

* Used to pass data in the URL.

* They start with a ? followed by key-value pairs.

```
app.get('/search', (req, res) => {
  const query = req.query.q;
  res.send(Search query: ${query});
});
```

Route Paths

Route paths are divided into two types:

1. Static Paths

2. Pattern Paths

1)Static Paths

A static path matches exactly what’s in the URL.

```
app.get('/about', (req, res) => {

res.send('Hello World!');

});
```

2) Pattern Paths

Strings and regular expressions to define route patterns

```
// Matches "/user/1", "/user/2", etc. only contains digits

app.get('/user/:id(\\d+)', (req, res) => {

res.send(req.params.id)

});

// Matches "/abcd", "/abbcd", "/abbbbbbbcd", etc.

app.get("/ab+cd",(req, res)=>{

  console.log(req.url)

  res.send("Hello World")

})
```

2) Route Handlers

1. Middleware functions can also act as route handlers.

2. Middleware functions can be chained to the route.

```
const express = require('express')

const app = express();

app.use((req, res, next) => {
```

```
  console.log(${req.method} ${req.url});
  next();  // Pass control to the next handler
});
app.get("/users",(req, res)=>{
  console.log(req.url)
  res.send("Hello World")
})
app.listen(5005, ()=>{
  console.log('started...')
})
```

Middlewares

1. Middleware route handlers are functions

2. This middleware acts as a logger.

3. It will execute during the lifecycle of a request to handle a specific task.

4. Middleware functions are called in sequence

5. It can be decided whether to pass control to the next middleware or terminate the response.

Middleware can perform various tasks:

1. Modifying the request objects

2. Modifying the response objects

3. Ending the request-response cycle

4. Calling the next middleware function in the stack

Syntax of Middleware

```
app.use([path], middlewareFunction);
```

app: An instance of Express.

path: (Optional)

A route path or a set of route paths to which the middleware function applies.

If not specified, the middleware applies to all paths.

middlewareFunction: The function that defines the middleware behavior.

Basic Middleware Function:

```
app.use((req, res, next)=>{
  console.log('Middleware executed');
        next();
});
app.get('/', (req, res) => {
  res.send('Hello World!');
});
```

req: The request object.

res: The response object.

next: When called, executes the next middleware function in the stack. If not called, the request will not proceed further.

Types of Middleware

Different types of Middleware in Express

Application-Level Middleware

Router-Level Middleware

Error-Handling Middleware

Built-in Middleware

Application-Level Middleware

These are bound to an instance of the Express application and are used across the entire app or specific routes.

```
app. use ((req, res, next )=>{

  next();

});

app.get('/', (req, res) => {

  res.send('Hello World!');

});


app.use('/users',(req, res, next)=>{

  next();

});

app.get('/users', (req, res) => {

  res.send('Hello World!');

});
```

2) Router-Level Middleware

Router-level middleware works similarly to application-level middleware.

Router object that can be used to create modular, mountable route handlers.

It is bound to an instance of express.router().

There are two ways to define routes using express.Router:

1. Grouped Routes Using express.Router

2. Chaining Route Methods

Grouped Routes Using express.Router

In this approach, we define routes in a separate router instance and then mount it in the main application.

```
const express = require('express');

const router = express.Router();


// Define routes inside the router

router.get('/user', (req, res) => {

  res.send('Get user');

});


router.post('/user', (req, res) => {

  res.send('Create user');

});


router.put('/user', (req, res) => {

  res.send('Update user');

});


// Mount the router in the main Express app

const app = express();

app.use('/', router);
```

```
app.listen(3000, () => console.log('Server running on port 3000'));
```

Chaining Route Methods

Instead of defining routes separately, we can chain multiple HTTP methods for a single route using router.route().

```
const express = require('express');

const router = express.Router();


router.route('/user')

 .get((req, res) => res.send('Get user'))

 .post((req, res) => res.send('Create user'))

 .put((req, res) => res.send('Update user'));


// Mount the router in the main Express app

const app = express();

app.use('/', router);


app.listen(3000, () => console.log('Server running on port 3000'));
```

3) Error-Handling Middleware

* It is a special kind of middleware that deals with errors.

* It provides a built-in mechanism to catch and handle errors that occur during the request-response cycle.

Syntax:

```
app.use((err, req, res, next) => {
```

```
  // error handling logic
});
```

Basic

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send('Something broke!');
});


const express = require('express');
const router = express.Router();


router.get('/user', (req, res, next) => {
  try {
    throw new Error('Something went wrong while getting the user!');
    res.send('Get user');
  } catch (err) {
    next(err);
  }
});


const app = express();
app.use('/', router);


// Error-handling middleware
app.use((err, req, res, next) => {
```

```
  console.error(err.stack);
  res.status(500).send({ message: err.message || 'Internal Server Error' });
});

app.listen(4000, () => console.log('Server running on port 4000'));
```

4) Built-in Middleware

Express provides several built-in middleware functions that help handle common tasks like logging, parsing request bodies, handling errors, and more.

Commonly used built-in middleware in Express.js:

* express.static()

* express.json()

* express.text()

* express.urlencoded()

express.raw()

express.static

The express.static() middleware is used to serve static files.

Static files include images, CSS, JS files, fonts, and other assets.

By default, files are served at the root (/).

Basic Usage

```
app.use(express.static('public'));
```

Any file inside the public directory will be automatically served.

You can access files directly by visiting http://localhost:3000/index.html

Serving Static Files from Multiple Directories

```
app.use(express.static('public'));
```

```
app.use(express.static('assets'));
```

If a file is not found in public/, it will look in assets/.

Serving Files Under /static Path

We can define a custom URL prefix using a virtual path.

```
app.use('/static', express.static('public'));
```

http://localhost:3000/static/index.html

http://localhost:3000/static/style.css

Restricting Access to Specific Files

We can use custom middleware to restrict access to specific static files:

```
// app.use('/static', express.static('public'));


app.use((req, res, next) => {
 if (req.url.startsWith('/static')) {
   return res.status(403).send('Access Forbidden');
 }
 next();
});
```

express.json

* It is used to parse incoming JSON payloads in requests

* It will make the data available in req.body.

* When a client sends a POST or PUT request with JSON data, the request body is not automatically parsed

* It’s extracts the JSON data and converts it into a JavaScript object accessible via req.body.

```
const express = require('express');
```

```
const app = express();

// Enabl JSON parsing middleware

app.use(express.json());

app.post('/data', (req, res) => {

 console.log(req.body); // The parsed JSON object

 res.send('JSON received successfully');

});
```

express.text

* It is used to parse incoming requests where the body contains plain text (text/plain content type).

* It converts the text body into a string and makes it available in req.body

* Postman -> Go to Body → Select raw → Set Content-Type to text/plain

```
app.use(express.text());

app.post('/text', (req, res) => {

 console.log('Received Text:', req.body);

 res.send(Text: ${req.body});

});
```

express.urlencoded

* When a client submits a form (<form> element in HTML), the data is sent in a URL-encoded format (application/x-www-form-urlencoded).

* express.urlencoded() parses the encoded data and converts it into a JavaScript object accessible via req.body.

```
app.use(express.urlencoded({ extended: true }));

app.post('/submit', (req, res) => {

        console.log(req.body);
```

```
        res.send('Form data received');
});
```

index.html

```
<form action="http://localhost:5500/submit" method="POST">
  <input type="text" name="name"/>
  <button type="submit">Submit</button>
</form>
```

express.raw

* It is used to parse raw request bodies into a buffer instead of converting them into a JavaScript object.

* It is useful when dealing with binary data such as file uploads, images, or raw text that should not be parsed automatically.

* Work with non-JSON, non-urlencoded data.

```
const express = require('express');
const fs = require('fs');
const path = require('path');
const app = express();

app.use(express.raw({ type: 'image/*', limit: '5mb' }));

app.post('/upload', (req, res) => {
 const filename = image-${Date.now()}.jpg;
 const filepath = path.join(__dirname, 'uploads', filename);

 fs.writeFile(filepath, req.body, (err) => {
```

```
  res.json({ message: 'Image uploaded successfully!'});
 });
});
```

EJS

EJS (Embedded JavaScript)

1. EJS (Embedded JavaScript) is a templating engine for Node.js and Express.js

2. It allows to embed JavaScript into HTML

3. It helps to render dynamic HTML pages on the server by embedding JavaScript logic into templates.

4. Instead of sending plain JSON responses, use a templating engine to generate structured HTML dynamically.

Why is required to use EJS?

*** Similar to HTML, with embedded JavaScript logic.**

*** Use JavaScript inside <% %>.**

*** Reuse common UI components like headers and footers.**

*** No need for a front-end framework.**

Install EJS with NPM.

npm install ejs

Create a Simple EJS Application

1. Steps to intigrate the ejs in express js

2. Create View Folder

Inside the project folder, create views folder

3. Create home page

Inside the views folder, create a file named home.ejs.

4. Add HTML Document

<h1>Welcome</h1>

5. Create the Express Server

6. Run the Application

node server.js

Create the Express Server

```
const express = require("express");

const app = express();

app.set("view engine", "ejs");// Tells Express to use EJS

app.get("/", (req, res) => {

  res.render("home");//Express will look for a file named home.ejs inside the views folder

});

app.listen(3000, () => {

  console.log("Server running on http://localhost:3000");

});
```

Basic esj Syntax

*<%- value %> * Outputs raw HTML (HTML to prevent XSS attacks).

```
<p><%- "<strong>Welcome</strong>" %></p>
```

*<%# comment %> * Comment in EJS (not shown in HTML output).

```
<%# About section %>
```

<% code %> Executes JavaScript code (like loops and conditions).

```
<ul>
```

```
  <% for(let i = 0; i < 5; i++) { %>
    <li>Item <%= i + 1 %></li>
  <% } %>
</ul>
```

Use <%= value %>

```
app.set("view engine", "ejs");
app.get("/", (req, res) => {
  res.render("home", { name: "ram" });
});
```

home.ejs

```
<h1>Hello <%= name %></h1>
```

Including Other Files

Include reusable components (like a header or footer)

```
<%- include("header") %>
<p>Homepage!</p>
<%- include("footer") %>
```

Displaying Dynamic Data from Express

Express Server

```
app.set("view engine", "ejs");
app.get("/", (req, res) => {
  res.render("home", {name: "ram", items:['Menu1', 'Menu2']})
});
```

home.ejs

```
<h1>Hello <%= name %></h1>

<ul>

  <% items.forEach(item => { %>

     <li><%= item %></li>

   <% }) %>

</ul>
```

home.ejs

```
<form action="/submit" method="POST">

     <label for="name">Name:</label>

     <input type="text" id="name" name="name" required>

     <label for="email">Email:</label>

     <input type="email" id="email" name="email" required>

     <button type="submit">Submit</button>

</form>
```

success.ejs

```
<h2>Form Submitted Successfully!</h2>

<p><strong>Name:</strong> <%= name %></p>

<p><strong>Email:</strong> <%= email %></p>

<a href="/">Go Back</a>
```

MONGODB

What is Database?

1. A database is an organized collection of data

2. It is stored and managed digital format

3. Used to store, retrieve, and manage information efficiently

4. It allows users and applications to interact with the data in a structured and meaningful way.

Relational Database Management System

Data Model (structured):

Tabular data model with rows and columns

Schema (fixed):

The structure of the data must be defined in advance

Scalability (vertically):

Scaling up by adding more resources (CPU, RAM) to a single server

Query Language (Structured Query Language):

Uses SQL for defining and manipulating data

Performance (performs better):

Generally performs better with complex queries and transactions

Examples:

MySQL, Microsoft SQL Server, PostgreSQL, Oracle

NoSQL Databases

Data Model (flexible):

Such as document, key-value, column-family, or graph models.

Schema (schema-less):

Allowing data to be inserted without a predefined structure

Scalability (horizontally):

Scaling out by adding more servers or nodes to the database cluster

Query Language (Lacks a standard query language):

Written in various ways depending on the NoSQL database(JSON)

Performance (faster):

Read/write operations in large-scale distributed systems

Examples:

MongoDB, DynamoDB, Cassandra, Redis

What is MongoDB?

1. MongoDB is a popular NoSQL database.

2. It stores data in a flexible, JSON-like format called BSON (Binary JSON).

3. MongoDB does not use tables or fixed schemas.

NoSQL:

1. MongoDB is a non-relational database

2. It doesn't use structured tables and rows like SQL databases

3. It stores data in documents

Databases and collections

In MongoDB, data is organized into databases and collections.

Database:

1. It is a container for collections.

2. A MongoDB instance can have multiple databases.

Collection:

1. It is a group of MongoDB documents.

2. It’s similar to a table in a relational database

3. Collections don't enforce a strict schema.

4. Collection can have different fields (key-value pairs).

Documents in MongoDB

MongoDB stores data as documents

* Those are similar to JSON objects

* A document is a set of key-value pairs

* Documents are stored inside collections

Document structure:

Documents are made up of key-value pairs, where keys are strings, and values can be various data types, including strings, numbers, arrays, or even nested documents.

MongoDB Connections Atlas & Compass

Steps to Connect via MongoDB Atlas:

Create a MongoDB Atlas Account:

1. Go to the MongoDB website.

2. Click on the "Try Atlas Free" button.

3. Sign up using Google or create an account with your email.

4. Review and accept the Privacy Policy & Terms of Service, then click on the Submit button.

5. After submission, you will be redirected to the MongoDB Atlas Dashboard at cloud.mongodb.com.

Create a New Project

1. In the top-left corner of the dashboard, click on the "Projects" dropdown menu.

2. Select "New Project" from the options.

3. Provide a name for your project in the "Project Name" field.

4. Click the "Next" button to proceed.

Set Up a New Cluster

1. After creating the project, you will be prompted to set up a new cluster.

2. Follow the steps to choose your cluster configuration, cloud provider, region, and tier based on your needs.

3. Click "Create Deployment" to launch your new cluster.

Configure Cluster and Database Access:

1. Once your cluster is created, click on "Database Access" under the Security tab in the left-hand sidebar.

2. Add a new database user with a username and password for connecting to the cluster.

3. Click "Network Access" to whitelist your IP address, allowing you to connect to the cluster from your current location.

Connect MongoDB in a Node.js application

* Create a folder

* Open a terminal or command prompt cmd

* Type the following command: npm init -y

* Press Enter

* Install the package: npm install express

* Install the package: npm install dotenv

(or)

* If you install express, dotenv along with nodemon:

npm install express nodemon dotenv

Create a Basic Server

```
const express = require('express');

const dotEnv = require('dotenv')

const app = express();

app.listen(3000, () => {

 console.log(Server is running on http://localhost:3000);

});
```

Run the app node server.js

connect to MongoDB

1. Install the MongoDB Package npm install mongodb

2. Set Up and Create a MongoDB connection URI in .env file

MONGODB_URL=mongodb+srv://vinodh:[password]@cluster1.3yqqn.mongodb.net/?retryWrites=true&w=majority&appName=Cluster1

3. Create a connection script in your server file

```
const { MongoClient } = require('mongodb');

// Load environment variables from .env file

dotEnv.config();

const client = new MongoClient(process.env.MONGODB_URL);

async function connectToDatabase() {

 try{

   await client.connect();

   console.log('MongoDB connected');

 } catch(error){

   console.log('MongoDB connection failed', error);

 }

}
```

```
// Start the server
app.listen(PORT, () => {
        console.log(Server started on port ${PORT});
        connectToDatabase();
});
```

Insert Data in MongoDB

```
const express = require('express');
const dotEnv = require('dotenv');
const { MongoClient } = require('mongodb');
dotEnv.config(); // Load environment variables from .env file
const app = express();  const PORT = 5001;

const client = new MongoClient(process.env.MONGODB_URL);
// Connect to the MongoDB database
async function connectToDatabase() {
 try {
  // Connect to the MongoDB server
  await client.connect();
  console.log('MongoDB connected successfully');
  const db = client.db('first-database'); // Access database and collection
  const collection = db.collection('first-collection');
  // Insert sample data
  const insertResult = await collection.insertOne({name: 'Ram', age: 28});
  console.log('Inserted document:', insertResult);
```

```
 } catch (error) {
   console.log('MongoDB connection failed:', error);
 }
}
```

Updating Documents in MongoDB

1. Using Update Operators

MongoDB provides several update operators to modify fields:

$set: Updates or adds a field.

$unset: Removes a field from the document.

$inc: Increments a field's value.

$push: Adds an item to an Array.

$pull: Removes an Item from an Array

Example

```
collection. updateOne (
    { "name": "Ram" },
    { $set: { "grade": 'First' } }
)
```

2. Update a Single Document

Syntax

```
collection.updateOne(
  { <filter> },          // Filter to find the document
  { $set: { <field>: <value> } }  // Update operation
)
```

Example

```
const db = client.db('first-database');

const collection = db.collection('first-collection');

collection.updateOne(

    { "name": "Ram" }, // Filter to find the document

    { $set: { "name": "RamKumar" } }// Update operation

)
```

4. Replace a Document

Syntax

```
db.collection.replaceOne(

  { <filter> },

  { <newDocument> }  // New document

)
```

Example

```
collection.replaceOne(

    { "name": "Ramm" },

    {

      "name": "Ram",

      "age": 20,

      "DOB": '06/07/2000',

      "marks": [50, 60 ,80]

    }

   )
```

Deleting Documents in MongoDB

Delete a Single Document:

Syntax collection.deleteOne({ <filter> })

Example collection.deleteOne({ "name": "Hanuman" })

Delete Multiple Documents:

Syntax collection.deleteMany({ <filter> })

Example collection.deleteMany({ "name": "Hanuman" })

Delete All Documents in a Collection

Syntax collection.deleteMany({})

Example collection.deleteMany({})

Drop a Collection

Syntax collection.drop()

Example collection.drop()

Querying Data in MongoDB

Querying in MongoDB involves retrieving data from your collections using various filters and conditions.

Find All Documents:

The find() method retrieves all documents from a collection.

Syntax: collection.find({})

Example: users.find({})

```
const query = {age: { $gt: 28 }}; // no filter

const options = {

    projection: { _id: 0, name: 1, age: 1 },// Optional: Which fields to return

    // sort: { age: -1 }, // Optional: descending order

    // limit: 5, // Optional
```

```
};
const documents = await collection.find(query, options).toArray();
console.log("Documents found:", documents);
```

Operators

Comparison Operators

$eq: Equal to

$ne: Not equal to

$gt: Greater than

$gte: Greater than or equal to

$lt: Less than

$lte: Less than or equal to

Logical Operators

$and

$or

$not

$nor

$eq (Equal to): Finds documents where a field is equal to a specified value.

```
{ age: { $eq: 25 } }
```

$ne (Not equal to): Finds documents where a field is not equal to a specified value.

```
{ age: { $ne: 25 } }
```

$gt (Greater than): Finds documents where a field is greater than a specified value.

{ age: { $gt: 25 } }

$gte (Greater than or equal to): Finds documents where a field is greater than or equal to a specified value.

{ age: { $gte: 25 } }

$lt (Less than): Finds documents where a field is less than a specified value.

{ age: { $lt: 25 } }

$lte (Less than or equal to): Finds documents where a field is less than or equal to a specified value.

{ age: { $lte: 25 } }

$and: Joins query clauses with a logical AND.

{ $and: [{ age: { $gt: 20 } }, { age: { $lt: 30 } }] }

$or: Joins query clauses with a logical OR.

{ $or: [{ name: 'Ram' }] }

$not: Inverts the effect of a query expression.

{ age: { $not: { $gte: 30 } } }

$nor: Joins query clauses with a logical NOR.

{ $nor: [{ age: { $gt: 30 } }, { name: 'Ram' }] }

Mongoose

1. Mongoose is an ODM (Object Data Modeling) library for MongoDB and Node.js.

2. It provides schema-based solutions for interacting with MongoDB

3. It makes it easier to handle database operations.

Install the Mongoose Package: npm install mongoose

connect to MongoDB using Mongoose

1. Set Up and Create a MongoDB connection URI in .env file

MONGODB_URL=mongodb+srv://vinodh: [password] @cluster1.3yqqn.mongodb.net/ [databasename] ?retryWrites=true

2. Script in your server file

```
const express = require('express');

const mongoose = require('mongoose');

const dotEnv = require('dotenv');

const app = express();

app.use(express.json());

dotEnv.config();


mongoose.connect(${process.env.MONGO_URL}, {

  useNewUrlParser: true,

  useUnifiedTopology: true

}).then(() => console.log('MongoDB Connected'))

 .catch(err => console.error(err));


app.listen(3000, () => console.log('Server running on port 3000'));


// Define a Schema
```

```
const userSchema = new mongoose.Schema({
  name: String,
  age: Number
});
/ / Create a Model
const User = mongoose.model('users', userSchema);

// Insert Data
const newUser = new User({ name: 'John', age: 30 });

newUser.save()
  .then(() => console.log('User Saved'))
  .catch(err => console.error(err));
```

Postman adds data using Mongoose

```
app.post('/submit',async (req, res)=>{
  const newUser = new User(req.body);
  await newUser.save()
  res.status(201).json({ message: 'User Saved', user: newUser });
})
app.get('/users', async (req, res) => {
  const users = await User.find();
  res.send(users);
});
```

Create Application Architecture

1. modesl/userModel.js

```
const mongoose = require('mongoose')

const userSchema= new mongoose.Schema({

  name: String,

  age: Number

})

const userModel =  mongoose.model('users', userSchema)

module.exports = userModel;
```

userController.js

```
const express = require('express')

const userRoutes = express.Router();

const userModel = require('../models/users')


const getAllUsers =('/',async (req, res)=>{

  const results = await userModel.find();

  res.send(results)

})

const createUser =('/',async (req, res)=>{

  const singleUser = new userModel(req.body)

  const newrecord = await singleUser.save();

  res.status(201).send(newrecord)

})

const updateUser =('/:id',async (req, res)=>{
```

```
  const {id} = req.params;
  const existingUser = await userModel.findById(id)
  existingUser.age = req.body.age
  const newrecord = await existingUser.save();
  res.status(200).json({ message: "User updated successfully", newrecord });
})
const deleteUser =('/:id',async (req, res)=>{
  const {id} = req.params;
  const deletedUser = await userModel.findByIdAndDelete(id)
  res.status(200).json({ message: "User deleted successfully", deletedUser });
})

module.exports = { getAllUsers, createUser, updateUser, deleteUser };
```

3. routes/userRoutes.js

```
const express = require('express');
const { getAllUsers, createUser, updateUser, deleteUser } = require('../controllers/users');
const userRoutes = express.Router();

userRoutes.get('/', getAllUsers);
userRoutes.post('/', createUser);
userRoutes.put('/:id', updateUser);
userRoutes.delete('/:id', deleteUser);

module.exports = userRoutes;
```

4. server.js

```
const express = require('express')

const app = express()

const mongoose = require('mongoose')

const dotenv = require('dotenv')

dotenv.config();

const userRoutes = require('./routes/users')

app.use(express.json())

app.use('/users',userRoutes)


const connectDB=async()=>{

  await mongoose.connect(${process.env.MONGO_URL})

  .then(()=>console.log('DB connected'))

  .catch((err)=>console.log(err))

}


app.listen(3000, ()=>{

  console.log('Server is running')

  connectDB();

})
```